

B1GMB42 Software Manual

IndianaDell workspace

2026-07-09

B1GMB42 Software Manual

Machine: Dell Precision Tower 5810 (B1GMB42)

Hostname: Tower5810

OS: Ubuntu 26.04 LTS (resolute)

Workspace: ~/Documents/IndianaDell

Companion hardware manual: B1GMB42-slot-port-inventory.md
(slots, GPUs, storage, PERC, ports)

Lab host Thumper (NVIDIA GPUs, power/clock locks): docs/thumper-gpu.md

This manual documents every **host-facing install** the IndianaDell workspace provides: apt packages, rustup, Python venvs, built tools, Flatpak apps, GNOME preferences, Plymouth themes, optional GPU/ROCm tooling, ZFS recovery, Ventoy live persistence, and GitHub sync. Each chapter covers one topic using the same structure:

1. What gets installed
2. How it is installed
3. How to verify
4. How to customize
5. What bin/rebuild-machine does and does not do

Build PDFs: bin/build-all-docs (all manuals) or bin/build-software-manual (this book only).

Quick reference: docs/features-available.md (cheat sheet, not a replacement for this manual).

GitHub: <https://github.com/webaugur/IndianaDell> (private)

Supersedes: flat B1GMB42-software-inventory.md (now a stub with links here).

Chapter 1 — Introduction

What this workspace installs

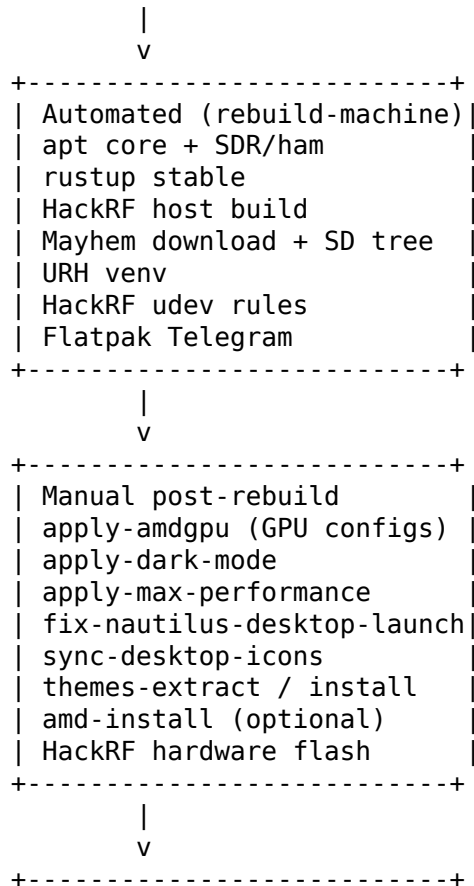
IndianaDell is a **software restoration toolkit** for Tower5810 after a fresh Ubuntu 26.04 install. It does not partition disks, configure ZFS, flash BIOS, or install Windows. It restores the development, SDR, ham radio, HackRF/Mayhem, documentation, and workstation utility stack documented in the chapters that follow.

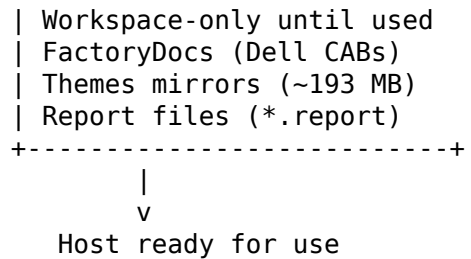
The workspace lives at `~/Documents/IndianaDell`. Copy or clone it before running `bin/rebuild-machine`.

Install layering

Software arrives in three layers. Understanding the order prevents skipped steps after a reinstall.

Fresh Ubuntu 26.04





Automated steps run via `bin/rebuild-machine` (see Chapter 2). **Manual** steps are intentional: GPU session files, GNOME gsettings, Plymouth overlay, and hardware flashing need user context or `sudo` at the right time. **Workspace-only** content stays in the repo until you invoke the matching `bin/launcher`.

Workspace vs host paths

Location	Role
<code>~/Documents/IndianaDell/</code>	Source of truth for scripts, themes, HackRF assets
<code>/usr/</code>	Apt-installed binaries, Plymouth themes, udev rules (after apply)
<code>~/ .cargo/</code>	Rust toolchain (rustup)
<code>hackrf/venv-urh/</code>	Universal Radio Hacker Python venv
<code>hackrf/build/</code>	HackRF host tools built from source
<code>hackrf/local/</code>	Optional CMAKE_INSTALL_PREFIX for built libhackrf
<code>Themes/*/mirror/</code>	Frozen copies of apt-owned theme files
<code>FactoryDocs/</code>	Dell vendor packages (not auto-installed to host)

Reading guide

If you need...	Read
Full restore after reinstall	Ch. 2 + Ch. 3
Python, Rust, pandoc	Ch. 4
Boot/login/desktop look	Ch. 5 + Ch. 7

If you need...	Read
FirePro GPUs, ROCm	Ch. 6
GNU Radio, gqrx, SoapySDR	Ch. 8
fldigi, WSJT-X, CHIRP	Ch. 9
HackRF, Mayhem, URH	Ch. 10
Telegram	Ch. 11
iotest, dellmerge	Ch. 12
Dell driver CABs	Ch. 13
Known gaps	Ch. 14
Ventoy live persistence (Uncle Wiggly + PNY rebuild stick), Grok, secrets	Ch. 15
ZFS rpool / bpool recovery	Ch. 2 + docs/B1GMB42- zfs-recovery.md (+ Ch. 15 live boot)
/etc/default/zfs force import	Ch. 2 / ZFS recovery manual — ZPOOL_IMPORT_OPTS="- f"
All bin/ commands	Appendix A
All apt package names	Appendix B

PATH and launchers

IndianaDell bin/ and scripts/ directories are prepended to PATH via `~/.config/indianadell/path.sh` (sourced from `~/.bashrc`). Project tools override same-named system binaries.

Related documents

- **Hardware:** B1GMB42-slot-port-inventory.md + PDF — GPUs, PERC, bays, ports
- **ZFS recovery:** docs/B1GMB42-zfs-recovery.md + PDF — live-media rpool/bpool chroot
- **PERC IT flash:** docs/B1GMB42-perc-it-flash.md — H710 FreeDOS/Wiggly path
- **Themes deep-dive:** Themes/README.md and per-folder READMEs
- **HackRF inventory:** hackrf/MANIFEST.txt
- **Apt snapshots:** apt-full-manifest.txt, apt-hamradio-dev-manifest.txt
- **Rebuild log:** scripts/rebuild/last-run.log

Chapter 2 — Rebuild and Recovery

What gets installed

`bin/rebuild-machine` restores the **workstation** software stack (core apt, rustup, Flatpak Telegram). The **SDR / ham / HackRF** stack is installed from **DragonSDR** when `~/Documents/DragonSDR` is present (`bin/install-dragonsdr`).

How it is installed

```
cd ~/Documents/IndianaDell
chmod +x bin/* scripts/rebuild/*.sh
bin/rebuild-machine           # full restore (includes DragonSDR suite if p
bin/rebuild-machine --verify-only # check only, no installs
bin/install-dragonsdr         # SDR suite alone
```

Environment overrides:

Variable	Effect
SKIP_TELEGRAM=1	Skip Flatpak Telegram install
SKIP_DRAGONSDR=1	Skip SDR suite install/verify
SKIP_HACKRF_BUILD=1	Forwarded to DragonSDR suite (skip cmake host build)
SKIP_HAM=1	Forwarded to DragonSDR (skip desktop ham apps)
DRAGONSDR_ROOT=...	Override suite path (default ~/Documents/DragonSDR)

Phases (from `scripts/rebuild/rebuild-machine.sh`):

Phase	Action
1	apt-get update
2	Install APT_CORE — build, Python, docs, GPU utils, flatpak, gh
3	Flatpak remote + org.telegram.desktop (unless skipped)
4	rustup stable if rustc missing
5	DragonSDR install-suite (apt SDR/ham + HackRF/Mayhem/URH) unless skipped
6	chmod bin/ and scripts

Phase	Action
7	Regenerate apt manifests; run verify_stack

Log file: scripts/rebuild/last-run.log

How to verify

```
bin/rebuild-machine --verify-only
bin/install-dragonsdr --verify-only
```

verify_stack checks:

- Every package in APT_CORE via dpkg-query
- Commands: rustc, cargo, pandoc, xelatex, vkcube
- Launchers: dellmerge, gpu-stress, iotest, apply-amdgpu, rebuild-machine
- DragonSDR suite (unless SKIP_DRAGONSDR=1 or suite missing)
- Flatpak Telegram (unless SKIP_TELEGRAM=1)

Exit code 0 means all checks passed.

How to customize

- **Add workstation apt packages:** Edit scripts/rebuild/package-lists.sh, update Appendix B, re-run rebuild.
- **Add SDR/ham packages:** Edit ~/Documents/DragonSDR/tools/package-lists.sh, re-run bin/install-dragonsdr.
- **Pin Mayhem version:** Edit DragonSDR/hackrf/scripts/download-mayhem.sh.
- **Skip heavy steps:** Use SKIP_* env vars for CI or partial recovery.

What rebuild does / does not do

Rebuild does	Rebuild does not
apt install APT_CORE	Partition disks or ZFS
rustup; DragonSDR suite when present	sudo bin/apply-amdgpu
Flatpak Telegram	GNOME prefs / themes by default
Regenerate apt manifests	Flash HackRF / PortaPack firmware
chmod workspace scripts	bin/amd-install (ROCm)
	Install FactoryDocs CABs to Windows

After a successful rebuild, continue with **Chapter 3 — Post-Rebuild Checklist**.

ZFS rpool / bpool recovery

Full guide: docs/B1GMB42-zfs-recovery.md + B1GMB42-zfs-recovery.pdf (also on **DOSBOOT** IndianaDell/recovery/).

Required on the installed system: /etc/default/zfs must set force import so boot can recover after export or unclean shutdown:

```
# /etc/default/zfs
ZPOOL_IMPORT_OPTS="-f"
```

Verify anytime: `grep ZPOOL_IMPORT_OPTS /etc/default/zfs`. One-shot boot alternative: kernel cmdline `zfsforce=1`.

Quick (with scripts, from Ventoy live):

```
sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell # or DOSBOOT/IndianaDell/recovery
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
```

Chapter 3 — Post-Rebuild Checklist

bin/rebuild-machine intentionally stops before steps that need a logged-in desktop, a reboot, or hardware attached. Run this checklist once per fresh install.

1. GPU session configuration

Tower5810 has three AMD FirePro cards (W5000/W5100). Multi-GPU Wayland/X11 configs live in etc/.

```
cd ~/Documents/IndianaDell
sudo bin/apply-amdgpu
sudo reboot
```

Verify after reboot: `echo $WAYLAND_DISPLAY, glxinfo -B, vkcube` on each display if needed.

See Chapter 6 for ROCm (bin/amd-install) — optional and not supported for ML on these GPUs.

2. GNOME session preferences

Run as the **desktop user** (not root):

```
bin/apply-dark-mode
bin/apply-max-performance
bin/fix-nautilus-desktop-launch
bin/sync-desktop-icons
```

```
# Yaru-dark GTK, shell, icons, GDM greeter
# no suspend, dimming, or night light
# Nautilus 50+: double-click .desktop launcher
# Nautilus 50+: show Icon= as file icon
```

Verify:

```
gsettings get org.gnome.desktop.interface color-scheme
powerprofilesctl get
bin/fix-nautilus-desktop-launch --status    # expect application/x-desktop → xdg-de
bin/sync-desktop-icons -v                  # sets metadata::custom-icon* from Ico
```

See Chapter 7 for every gsettings key touched, the Nautilus 50 “Allow Launching” note, and custom-icon metadata.

3. Boot splash (optional)

Default Ubuntu **bgrt** Plymouth theme shows Dell BGRT center + Ubuntu watermark. To customize:

```
bin/themes-extract                        # refresh mirrors + extract logos
# edit Themes/boot/overlay/watermark.png or background.png
sudo bin/themes-install-boot              # or --oem / --no-watermark
sudo reboot
```

Restore factory: `sudo bin/themes-restore-boot`

See Chapter 5.

4. HackRF hardware (when device is available)

```
bin/install-dragonsdr --verify-only      # suite present?
source bin/hackrf-env                     # PATH → ~/Documents/DragonSDR/hackrf
hackrf_info                              # should list board
bin/hackrf-flash-mayhem                   # extract USB flash bundle
# follow bundle README for DFU flash
bin/hackrf-prepare-sdcard                 # ensure SD tree is extracted
# copy ~/Documents/DragonSDR/hackrf/sd-card/mayhem-v2.4.0/* to FAT32 microSD root
```

Verify: `hackrf_info`, on-device Mayhem version, SD apps visible on PortaPack.

See Chapter 10 and `~/Documents/DragonSDR/README.md`.

5. Documentation PDFs

```
bin/build-all-docs                       # software manual + hardware + inventory PDF
# or:
bin/build-software-manual                 # this manual only
```

Outputs: `B1GMB42-software-manual.pdf`, `B1GMB42-slot-port-inventory.pdf`, `B1GMB42-software-inventory.pdf`.

6. Machine inventory baseline

```
bin/dellmerge > blgmb42.report
sudo bin/iotest                    # optional storage survey
bin/gpu-stress 60 vkcube           # optional GPU smoke test
```

7. FactoryDocs recovery (optional)

Only 19 of 101 pre-crash Dell packages are on disk. Re-download per FactoryDocs/README.md and MANIFEST-pre-crash.txt. These are **workspace archives**, not installed by rebuild.

See Chapter 13.

8. ZFS force import (required on this host)

After any reinstall or recovery chroot, confirm the installed system will force-import pools at boot:

```
grep '^ZPOOL_IMPORT_OPTS' /etc/default/zfs
# must show: ZPOOL_IMPORT_OPTS="-f"
```

If missing, set it in /etc/default/zfs, then `sudo update-initramfs -c -k all`. Without this, boot can hang after a recovery export or unclean shutdown. See Chapter 2 and docs/B1GMB42-zfs-recovery.md.

Quick verification block

```
cd ~/Documents/IndianaDell
bin/rebuild-machine --verify-only
grep '^ZPOOL_IMPORT_OPTS' /etc/default/zfs # expect "-f"
source bin/hackrf-env
. ~/.cargo/env && rustc --version
gnuradio-config-info --version
bin/urh --version
```

Summary table

Step	Command	Reboot?
GPU configs	sudo bin/apply-amdgpu	Yes
Dark mode	bin/apply-dark-mode	No
Max performance	bin/apply-max- performance	No

Step	Command	Reboot?
Nautilus 50 .desktop launch	bin/fix-nautilus- desktop-launch	No
Nautilus 50 .desktop icons	bin/sync-desktop- icons	No
Custom boot	sudo bin/themes- install-boot	Yes
HackRF flash	bin/hackrf-flash- mayhem + DFU	Maybe
ROCM (optional)	bin/amd-install	Yes
All doc PDFs	bin/build-all-docs	No
ZFS force import	check /etc/default/zfs □ ZPOOL_IMPORT_OPTS=" - f"	If initramfs updated
Ventoy persistence seed	~/bin/seed-ventoy- persistence.sh	No

Chapter 4 — Development Toolchain

What gets installed

Component	Version / path	Install source
Python pip, venv	3.14 (system) apt	Ubuntu base + apt python3-pip, python3-venv
numpy, scipy, matplotlib PyQt5	apt apt	science stack GNU Radio Companion, some tools
Rust C/C++ build	1.96.1 stable apt	rustup to ~/.cargo/bin build-essential, cmake, pkg-config
Clang/LLVM	apt	bindgen-style Rust, native tooling
SSL/USB/FFTW/Volk ARM cross-compile	apt dev libs apt	SDR and native builds PortaPack Mayhem firmware (gcc-arm-none-eabi)
pandoc + XeLaTeX	apt	Manual PDF generation
Git, curl, wget	apt	repos and downloads

Component	Version / path	Install source
GitHub CLI (gh)	2.46 (apt)	HTTPS/token fallback; gh auth login for API access

Python bindings verified on this host: gnuradio, SoapySDR, Hamlib (capital H in Python).

Project venv: URH lives in ~/Documents/DragonSDR/hackrf/venv-urh/ (Chapter 10), not system-wide.

How it is installed

Apt (rebuild Phase 2): packages listed in Appendix B under “Development” and shared dev libraries.

Rust (rebuild Phase 5):

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y --default-source ~/.cargo/env
```

Rebuild skips rustup if rustc is already on PATH.

How to verify

```
python3 --version
python3 -c "import numpy, scipy, matplotlib; print('OK!')"
. ~/.cargo/env && rustc --version && cargo --version
cmake --version | head -1
arm-none-eabi-gcc --version | head -1
pandoc --version | head -1
xelatex --version | head -1
```

How to customize

- **New Python project venv:** `python3 -m venv myproject/.venv`
`&& . myproject/.venv/bin/activate`
- **Rust toolchain:** `rustup toolchain install stable, rustup component add clippy`
- **Per-project Rust SDR crates:** `cargo add rtlsdr etc.` — no workspace scaffold ships with IndianaDell
- **Manual PDFs:** `bin/build-all-docs` (all docs) or `bin/build-software-manual` (software manual only)

What rebuild does / does not do

Does	Does not
Install all dev apt packages	Create application-specific venvs beyond URH
Install rustup if missing	Pin a non-stable Rust toolchain
chmod workspace scripts	Install IDE or editor plugins

Chapter 5 — Themes (Boot, Login, Desktop)

What gets installed

The **Themes/** module (~194 MB with mirrors) documents and customizes three visual stages:

Stage	What you see	Apt packages	Workspace folder
Boot	Dell BGRT center + spinner + Ubuntu watermark	plymouth, plymouth-theme-spinner, ...	Themes/boot/
Login	GDM on GNOME Shell	gdm3, gnome-shell, yaru-theme-gnome-shell	Themes/login/
Desktop	Yaru GTK, icons, shell	yaru-theme-gtk, yaru-theme-icon, ...	Themes/desktop/

Active Plymouth theme: `bgrt at /usr/share/plymouth/themes/bgrt/bgrt.plymouth`

Extracted boot logos:

- Themes/boot/extracted/bgrt-firmware-oem.png — Dell from UEFI BGRT (`/sys/firmware/acpi/bgrt/image`)
- Themes/boot/extracted/ubuntu-watermark-dark.png — bottom Ubuntu text

Custom Plymouth install target: `indianadell` theme under `/usr/share/plymouth/themes/indianadell`

Each subfolder has its own `README.md` and `apt-packages.txt`. See `Themes/MANIFEST.txt` for a one-page map.

How it is installed

Themes are **not** applied by `bin/rebuild-machine`. Use launchers:

```
bin/themes-extract          # snapshot apt-owned files + extract logos (~193 MB)
sudo bin/themes-install-boot # install custom Plymouth from boot/overlay/
sudo bin/themes-restore-boot # revert to stock bgrt
bin/apply-dark-mode         # login + desktop dark (Chapter 7)
```

Boot overlay workflow:

```
cp my-logo.png Themes/boot/overlay/watermark.png # bottom Ubuntu text only
sudo bin/themes-install-boot

cp my-splash.png Themes/boot/overlay/background.png
sudo bin/themes-install-boot --oem # replace center (disable BGRT)

sudo bin/themes-install-boot --no-watermark # hide bottom logo
```

Every Plymouth install runs `update-initramfs -u` — **reboot required** to preview.

How to verify

```
plymouth-set-default-theme -l
plymouth-set-default-theme # shows active theme
ls Themes/boot/extracted/
gsettings get org.gnome.desktop.interface gtk-theme # after apply-dark-mode
```

How to customize

Deep documentation lives in module READMEs — this chapter summarizes:

- **Boot:** Themes/boot/README.md — overlay/, stock/, indianadell/ plymouth definition
- **Login:** Themes/login/README.md — GDM greeter assets mirrored from apt
- **Desktop:** Themes/desktop/README.md — GTK/icon/shell Yaru variants

Center Dell logo comes from BIOS BGRT firmware, not an Ubuntu file. Change it in BIOS setup or use `--oem` with your PNG.

What rebuild does / does not do

Does	Does not
Install plymouth, gdm3, Yaru via apt (Phase 2–3)	Run themes-extract or themes-install-boot Set dark mode or GDM greeter prefs Copy overlay PNGs into initramfs

Does	Does not
------	----------

Run Chapter 3 checklist items 2 and 3 after rebuild.

Chapter 6 — GPU and Display

What gets installed

Component	Source	Purpose
vulkan-tools, mesa-utils, clinfo etc/ multi-GPU configs	apt (APT_CORE) workspace	Vulkan/OpenGL/OpenCL diagnostics Wayland, X11, udev, GDM tweaks
amd-radeon/ scripts	workspace	Optional ROCm driver install
bin/gpu-stress	workspace	3-GPU Vulkan smoke test

Hardware (this machine): 2x AMD FirePro W5000 + 1x FirePro W5100. Vulkan and OpenCL work for graphics/compute smoke tests. **ROCm ML/HIP is not supported** on these cards (see Chapter 14).

How it is installed

Apt (automated): GPU utility packages install during rebuild Phase 2.

Session configs (manual):

```
sudo bin/apply-amdgpu    # runs etc/apply.sh
sudo reboot
```

etc/apply.sh installs:

- etc/environment.d/99-amdgpu-wayland.conf
- etc/X11/xorg.conf.d/20-amdgpu-multi-gpu.conf
- etc/modprobe.d/amdgpu-multigpu.conf (runpm=0)
- etc/udev/rules.d/99-amdgpu-multigpu.rules (tags + DPM performance hook)
- etc/amdgpu-set-dpm-performance.sh → /usr/local/sbin/indiana-amdgpu-dpm-performance
- etc/profile.d/amdgpu-multigpu.sh
- etc/gdm3/custom.conf (if present)

DPM performance (all cards): on a desktop workstation every amdgpu is pinned to max clocks — not only the display GPU. `apply-amdgpu` runs the helper immediately; `udev` re-applies when cards appear at boot.

```
# verify
for c in /sys/class/drm/card[0-9]/device; do
    [[ -f $c/power_dpm_force_performance_level ]] || continue
    echo "$(basename $(dirname $c)): level=$(cat $c/power_dpm_force_performance_level)"
done
# expect: level=high state=performance on card1..card3
```

Tradeoff: slightly higher idle power/heat/fan noise vs auto/balanced.
 Revert by editing the `udev` rule / helper and re-running `sudo bin/apply-amdgpu`, or manually `echo auto | sudo tee .../power_dpm_force_performance_level`.

Optional ROCm:

```
bin/amd-preflight      # check prerequisites
bin/amd-install        # full driver stack from amd-radeon/
bin/amd-verify
bin/amd-uninstall      # remove if needed
```

How to verify

```
vkcube                # Vulkan cube (per display)
clinfo | head -30      # OpenCL platforms/devices
glxinfo -B             # OpenGL renderer
bin/gpu-stress 60 vkcube # stress all GPUs ~60s
lspci -nn | grep -i vga
```

How to customize

- Edit files under `etc/` before re-running `sudo bin/apply-amdgpu`
- ROCm install scripts and README in `amd-radeon/` — machine-specific; read preflight output
- Hardware details: `B1GMB42-slot-port-inventory.md` Video section

What rebuild does / does not do

Does	Does not
Install mesa-utils, vulkan-tools, clinfo	Run <code>apply-amdgpu</code>
Ensure <code>bin/gpu-stress</code> is executable	Install ROCm
	Configure monitor layout (use GNOME Settings)

Required post-rebuild: `sudo bin/apply-amdgpu + reboot` (Chapter 3).

Lab host note — Thumper (NVIDIA)

This chapter is **Tower5810 / AMD only**. The lab host **thumper.local** (Dell Precision T5610) runs **NVIDIA TITAN Xp** and is **not** configured with `apply-amdgpu`.

Full Thumper GPU doc (inventory, dual-card plan, **power/clock locking**, LingBot-Map pointer):

□ `docs/thumper-gpu.md`

Summary: a second TITAN Xp with a weak cooler can be limited with `nvidia-smi -pl / -lgc` (and a oneshot systemd unit). Prefer `CUDA_VISIBLE_DEVICES` so heavy jobs stay on the well-cooled card. DragonSDR LingBot-Map lives under `~/Data/lingbot-map` on Thumper — see also `~/Documents/DragonSDR/tools/lingbot-map/README.md`.

Chapter 7 — GNOME Session

What gets installed

GNOME desktop preferences applied via `gsettings` (user session) and optionally GDM (login greeter). No apt packages beyond what Ubuntu desktop already provides (`gdm3`, `gnome-shell`, Yaru themes).

Scripts:

- `scripts/gnome/apply-dark-mode.sh` via `bin/apply-dark-mode`
- `scripts/gnome/apply-max-performance.sh` via `bin/apply-max-performance`
- `scripts/gnome/fix-nautilus-desktop-launch.sh` via `bin/fix-nautilus-desktop-launch`
- `scripts/gnome/sync-desktop-icons.sh` via `bin/sync-desktop-icons`

How it is installed

Run as the **logged-in desktop user**, not root:

`bin/apply-dark-mode`

`bin/apply-max-performance`

`bin/fix-nautilus-desktop-launch`

`bin/sync-desktop-icons`

Nautilus 50+ .desktop double-click launch
Nautilus 50+ show Icon= as file icon

apply-dark-mode

Sets:

Schema	Key	Value
org.gnome.desktop.interface	color-scheme	prefer-dark
org.gnome.desktop.interface	gtk-theme	Yaru-dark (override: GTK_THEME=)
org.gnome.desktop.interface	icon-theme	Yaru-dark
org.gnome.shell.ubuntu	color-scheme	prefer-dark
org.gnome.desktop.wm.preferences	themes	Yaru-dark
org.gnome.settings-daemon.plugins.color	night-light	disabled

With APPLY_GDM=1 (default), also sets GDM greeter dark via `sudo dbus-run-session gsettings ....`

apply-max-performance

Sets:

- Power plugin: no suspend on AC/battery, no idle dim, lid close does nothing
- Session idle delay: 0
- Screensaver: no blanking or lock on idle
- Night light: off
- `powerprofilesctl set performance` when available

Nautilus 50 — “Allow Launching” removed

Change (GNOME Files / Nautilus 50, Ubuntu 26.04): Nautilus no longer runs FreeDesktop .desktop files itself. The old “Allow Launching” / trusted-launcher path is gone (security). Double-click falls through to the default handler for MIME type application/x-desktop, which is often a text editor (gedit / gnome-text-editor). So double-clicking SDRPlusPlus.desktop (or any app launcher on disk) **edits** the file instead of **starting** the app.

Fix: register a small MIME handler that launches the entry via `gio launch:`

Piece	Path
Handler app	<code>~/.local/share/applications/xdg-desktop-launch.desktop</code>
Wrapper script	<code>~/.local/bin/xdg-desktop-launch</code>

Piece	Path
MIME default	application/x-desktop xdg-desktop-launch.desktop

Double-click path after install: **Files** → **xdg-open** → **wrapper** → **gio launch** → **your app**.

```
bin/fix-nautilus-desktop-launch # install / reinstall
bin/fix-nautilus-desktop-launch --status # check MIME + files
bin/fix-nautilus-desktop-launch --uninstall # remove handler
```

Portable: the script is self-contained. Copy scripts/gnome/fix-nautilus-desktop-launch.sh to any Ubuntu/GNOME box and run as the desktop user (no root, no IndianaDell tree required).

CLI verify:

```
xdg-mime query default application/x-desktop # expect xdg-desktop-launch.desktop
xdg-open /path/to/App.desktop # should start the app
~/.local/bin/xdg-desktop-launch /path/to/App.desktop
```

Nautilus 50 — generic .desktop icons

Change: even after launch works, Nautilus 50+ still shows the generic application-x-desktop MIME icon in the file view. It **ignores** the FreeDesktop Icon= field on the .desktop file for the list/icon view. Icons in the GNOME Shell app grid (from XDG application menus) are a different path and usually look correct; this problem is about **Files** showing launcher files as plain documents.

Fix: set GIO metadata that Nautilus still honors:

Attribute	When used	Example
metadata::custom-icon	Icon= is an absolute or relative file path	file:///home/user/Applications/sdrpp.pr
metadata::custom-icon-name	Icon= is a theme icon name	utilities-terminal

The script reads the first Icon= under [Desktop Entry] only (not Icon[lang]=, not other groups), then:

1. Absolute path (/...) → metadata::custom-icon with a file:// URI; clear custom-icon
2. Relative path (foo/bar.png) → resolve relative to the .desktop file's directory; same as absolute
3. Theme name (no /) → metadata::custom-icon-name; clear custom-icon

4. Missing icon file → warning and skip (does not fail the whole run)

Usage

```
bin/sync-desktop-icons           # scan default directories
bin/sync-desktop-icons -v        # log each set / skip / rename
bin/sync-desktop-icons --dry-run # print actions, no gio set / rename
bin/sync-desktop-icons --file PATH # one .desktop (inotify-friendly)
bin/sync-desktop-icons --dir DIR  # add/replace scan dir (repeatable)
bin/sync-desktop-icons --watch    # inotify loop (needs inotify-tools)
bin/sync-desktop-icons --clear-missing # unset custom-icon* if Icon= absent
bin/sync-desktop-icons --no-rename  # keep chrome-*-Default.desktop names
```

Default scan directories (when `--dir` is not used and `SYNC_DESKTOP_ICON_DIRS` is unset):

- `$HOME/.local/share/applications`
- `$HOME/Applications`
- `$HOME/Desktop`

Only **top-level** `*.desktop` files in each directory are processed (flat XDG apps layout and a personal Applications / Desktop folder). Nested trees are not walked.

Rename Chrome gibberish basenames

Chrome/Chromium PWAs create launchers like `chrome-lodlkdfmihgonocnmdehnfgiljnadcf-Default.desktop` while `Name=` is a short human label (X, YouTube). **By default** (`--rename`), if the file **and** its directory are **writable**, matching basenames are renamed to `${Name}.desktop` before icon metadata is applied.

Rule	Behavior
Pattern	<code>chrome-<id>-Default.desktop</code> , <code>chrome-<id>.desktop</code> (<code>id</code> ≤ 16 alnum); same for chromium-
Source of new name	First <code>Name=</code> under [Desktop Entry] only (not action groups, not <code>Name[lang]=</code>)
Not writable	Skip rename, warn, still try icon metadata
Target already exists	Skip rename, warn (no overwrite)
Disable	<code>--no-rename</code>

Example: `~/Desktop/chrome-lodlk...-Default.desktop` (`Name=X`) →
`~/Desktop/X.desktop`.

Environment:

Variable	Effect
SYNC_DESKTOP_ICON_DIRS	Colon-separated directory list (overrides defaults when set)

Dependencies: gio (libglib2.0-bin, already on Ubuntu desktop). --watch also needs inotifywait (inotify-tools).

Portable / PATH install: bin/sync-desktop-icons resolves its own path with readlink -f, so a symlink from ~/.local/bin/sync-desktop-icons into the repo still finds scripts/gnome/. The script itself needs only gio; copy scripts/gnome/sync-desktop-icons.sh alone if you want it off-tree.

Companion: run **after** bin/fix-nautilus-desktop-launch so double-click starts the app **and** the file view shows the right icon. Neither script replaces the other.

CLI verify:

```
bin/sync-desktop-icons -v
# pick a launcher you care about:
gio info -a metadata::custom-icon -a metadata::custom-icon-name \
"$HOME/Applications/SomeApp.desktop"
# open Files on that folder – icon should match Icon=
```

Watch mode example (re-apply when a launcher is saved or dropped into a scan dir):

```
bin/sync-desktop-icons --watch -v
# or a single-file handler:
inotifywait -m -e close_write,moved_to,create --include '\.desktop$' \
"$HOME/Applications" | while read -r dir _file; do
    bin/sync-desktop-icons --file "$dir$file"
done
```

How to verify

```
gsettings get org.gnome.desktop.interface color-scheme
gsettings get org.gnome.desktop.interface gtk-theme
gsettings get org.gnome.settings-daemon.plugins.power sleep-inactive-ac-type
powerprofilesctl get
bin/fix-nautilus-desktop-launch --status
bin/sync-desktop-icons -v
```

Log out and back in to confirm GDM greeter if dark mode was applied. Refresh or reopen the Files window after sync-desktop-icons if icons do not update immediately.

How to customize

- Override themes: `GTK_THEME=Yaru-viridian bin/apply-dark-mode`
- Skip GDM: `APPLY_GDM=0 bin/apply-dark-mode`
- Revert individual keys with `gsettings reset ...` or GNOME Settings app
- Login/desktop asset mirrors: `Themes/login/, Themes/desktop/`
- Remove .desktop launch fix: `bin/fix-nautilus-desktop-launch --uninstall`
- Extra icon scan dirs: `SYNC_DESKTOP_ICON_DIRS="$HOME/Apps:$HOME/Desktop" bin/sync-desktop-icons`
- Clear stale custom icons when Icon= is gone: `bin/sync-desktop-icons --clear-missing`
- Unset metadata on one file:
`gio set -t unset PATH metadata::custom-icon`
`gio set -t unset PATH metadata::custom-icon-name`

What rebuild does / does not do

Does	Does not
Install gdm3, gnome-shell, Yaru via apt	Change any gsettings Set performance power profile Install the Nautilus 50 .desktop MIME handler Run sync-desktop-icons or set custom-icon metadata

Run dark mode, max performance, fix-nautilus-desktop-launch, and sync-desktop-icons after every fresh install (Chapter 3).

Chapter 8 — GNU Radio and Desktop SDR

Ownership

GNU Radio, SoapySDR, and desktop SDR apps are installed by the **DragonSDR** suite, not by IndianaDell apt lists.

Item	Location
Suite install	<code>~/Documents/DragonSDR/bin/install-suite</code>
Package list	<code>~/Documents/DragonSDR/tools/package-lists.sh (APT_SDR)</code>
IndianaDell wrapper	<code>bin/install-dragonsdr</code>

Item	Location
------	----------

What gets installed

Component	Packages / path
GNU Radio Companion blocks	gnuradio, gnuradio-dev, gnuradio-doc gr-osmosdr, gr-limesdr, gr-fosphor, gr-air-modes, gr-hpsdr, gr-dab, gr-satellites
SoapySDR	libsoapysdr-dev, python3-soapysdr, modules
Hardware libs	RTL-SDR, HackRF, Airspy, bladeRF, Lime, UHD
Desktop apps	gqrx-sdr, quisk, inspectrum, hacktv

SoapySDR modules (typical host): HackRF, RTL-SDR (osmosdr), Airspy, bladeRF, Lime, MiriSDR, HydraSDR, PlutoSDR, Red Pitaya, remote, audio, UHD.

How it is installed

```
bin/install-dragonsdr          # full suite (apt + HackRF workspace)
# or only packages:
bin/install-dragonsdr --apt-only
```

Called automatically during bin/rebuild-machine when ~/Documents/DragonSDR is present (SKIP_DRAGONSDR=1 to skip).

Typical workflow:

```
g++ myflowgraph.grc           # compile Companion graph
gqrx                          # general receiver GUI
quisk                          # transceiver GUI
inspectrum capture.cf32       # visualize IQ files
```

For HackRF-specific host tools and URH, see Chapter 10.

How to verify

```
bin/install-dragonsdr --verify-only
gnuradio-config-info --version
python3 -c "import SoapySDR; print('SoapySDR OK')"
```

SoapySDRUtil --info

```
command -v gqrx
```

Chapter 9 — Ham Radio (Desktop)

Ownership

Desktop ham applications are part of the **DragonSDR** suite (APT_HAM in tools/package-lists.sh).

What gets installed

Application	Command	Apt package	Role
fldigi	fldigi	fldigi	Digital modes (PSK, RTTY, ...)
WSJT-X	wsjtx	wsjtx, wsjtx-data	FT8, JT65, weak-signal
CHIRP	chirpw, chirpc	chirp	Radio programming
direwolf	direwolf	direwolf	Sound-card TNC / APRS
gpredict	gpredict	gpredict	Satellite pass prediction
grig	grig	grig	Hamlib rig control GUI
xastir	xastir	xastir, xastir-data	APRS map client
Hamlib	API	libhamlib-dev, libhamlib-utils, python3-hamlib	Rig control library

How it is installed

```
bin/install-dragonsdr
# omit ham apps:
SKIP_HAM=1 bin/install-dragonsdr
```

xastir debconf: suite install preseeds xastir/install-setuid boolean false to avoid interactive hangs.

```
fldigi &
wsjtx &
chirpw &
direwolf -p
gpredict &
grig &
xastir &
```

How to verify

```
command -v fldigi wsjtx chirpw direwolf gpredict grig xastir
python3 -c "import Hamlib; print('Hamlib OK')"
```

```
bin/install-dragonsdr --verify-only
```

Configure rig control in each app via Hamlib model selection.

Chapter 10 — HackRF and PortaPack Mayhem

Ownership

The HackRF / PortaPack Mayhem workspace moved out of IndianaDell into **DragonSDR**:

Item	Path
Workspace	~/Documents/DragonSDR/hackrf/
Manifest	~/Documents/DragonSDR/hackrf/MANIFEST.txt
Suite install	~/Documents/DragonSDR/bin/install-suite
IndianaDell wrappers	bin/hackrf-*, bin/urh, bin/install-dragonsdr

What gets installed

Recommended firmware: PortaPack Mayhem v2.4.0

Apt packages

hackrf, hackrf-firmware, libhackrf-dev, hackrf-doc, inspectrum, hacktv, dfu-util, openocd, ARM GCC toolchain, plus GNU Radio/SoapySDR deps (Chapter 8).

Built from source (DragonSDR/hackrf/build/)

Tool	Notes
hackrf_sweep	Spectrum sweep
hackrf_info, hackrf_transfer, ...	Host utilities
libhackrf.so	Under hackrf/build/libhackrf/src/

Install prefix: hackrf/local/ (CMAKE_INSTALL_PREFIX).

Release assets (hackrf/releases/)

File	Purpose
FIRMWARE_mayhem_v2.4.0.zip	USB flash bundle
COPY_TO_SDCARD_hackrf_mayhem_v2.4.0	Portable microSD
no-world-map.zip	Portable
OCI_hackrf_mayhem_v2.4.0.webflasher	Web flasher image

Extracted SD tree: hackrf/sd-card/mayhem-v2.4.0/

Source repos (hackrf/repos/)

hackrf, mayhem-firmware (+ submodules), portapack-hackrf, urh, hacktv

Python venv

hackrf/venv-urh/ — Universal Radio Hacker. Launch: bin/urh (wrapper □ DragonSDR).

udev

hackrf/scripts/99-hackrf.rules □ /etc/udev/rules.d/

How it is installed

```
bin/install-dragonsdr # full suite
SKIP_HACKRF_BUILD=1 bin/install-dragonsdr
bin/install-dragonsdr --hackrf-only # workspace only (apt already done)
```

Manual (hardware present):

```

source bin/hackrf-env                # PATH → DragonSDR/hackrf/build
bin/hackrf-flash-mayhem
bin/hackrf-prepare-sdcard
bin/hackrf-build-mayhem              # compile Mayhem from source
bin/hackrf-download-mayhem
bin/urh

```

How to verify

```

bin/install-dragonsdr --verify-only
source bin/hackrf-env
hackrf_info
ls ~/Documents/DragonSDR/hackrf/sd-card/mayhem-v2.4.0/APPS | wc -l
bin/urh --version

```

Chapter 11 — Flatpak Applications

What gets installed

App	Flatpak ID	Version (this host)
Telegram	org.telegram.desktop	6.9.3

Runtime dependency: flatpak package from APT_CORE.

How it is installed

Rebuild Phase 4:

```

flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.flatpakrepo
flatpak install -y flathub org.telegram.desktop
Skip with SKIP_TELEGRAM=1 bin/rebuild-machine.

```

How to verify

```

flatpak list --app | grep telegram
flatpak run org.telegram.desktop --version 2>/dev/null || true
bin/rebuild-machine --verify-only

```

How to customize

```

flatpak update org.telegram.desktop
flatpak override --user org.telegram.desktop ... # permissions, env

```

bin/apply-dark-mode sets prefer-dark color scheme; Flatpak GTK4 apps pick this up via portal when supported.

What rebuild does / does not do

Does	Does not
Install flatpak via apt	Install SDRangel, SigDigger (not on Flathub)
Add flathub remote + Telegram	Pin Telegram to a specific commit
Treat Telegram miss as non-fatal on install (warns in log)	Install other Flatpak apps by default

Chapter 12 — Machine Utilities

What gets installed

Workspace scripts for inventory, storage benchmark, and GPU stress — no dedicated apt packages beyond shared GPU utils (mesa-utils, vulkan-tools).

Utility	Launcher	Script	Output
Dell inventory	bin/dellmerge	scripts/dell/dellmerge.sh	dellmerge*.report files
Storage survey	bin/iotest	scripts/storage/iotest.sh	IOintest (sudo)
GPU stress	bin/gpu-stress	scripts/gpu/gpuVulkan/EGL stress.sh	per GPU
EFI / BIOS timing	bin/efi-timing-suite	scripts/efi/efiB1GMB42.timing timing-suite.sh	B1GMB42.timing (sudo)

Example reports in workspace: blgmb42.report, B1GMB42.ioperf, B1GMB42.timing (from prior runs).

How it is installed

Scripts ship with the workspace. Rebuild Phase 10 runs chmod +x on bin/* and scripts/*/*.sh.

```
bin/dellmerge > blgmb42.report
sudo bin/iotest
bin/gpu-stress 60 vkcube
sudo bin/efi-timing-suite # before/after BIOS A/B changes
```

How to verify

```
[[ -x bin/dellmerge && -x bin/iotest && -x bin/gpu-stress && -x bin/efi-timing-suite &&
bin/rebuild-machine --verify-only # checks dellmerge, gpu-stress, iotest, apply
head -20 blgmb42.report 2>/dev/null || bin/dellmerge | head -20
```

How to customize

- Edit scripts/dell/dellmerge.sh to add inventory fields
- gpu-stress accepts duration and backend (vkcube default)
- iotest targets block devices — read script header before running on production pools
- efi-timing-suite writes a machine-local timing baseline; re-run after BIOS changes for A/B compare

What rebuild does / does not do

Does	Does not
chmod utility launchers	Run dellmerge, iotest, or efi-timing automatically
Verify launcher executables exist	Archive reports to a fixed path

Chapter 13 — FactoryDocs (Workspace Archive)

What gets installed

Nothing on the Linux host automatically. FactoryDocs is a sorted archive of Dell T5810 vendor support packages for Windows recovery, firmware, and reference — stored only in the workspace.

Metric	Pre-crash	Current
Sorted packages	101	19
GPU drivers (FirePro/Quadro)	Yes	Missing
PERC H710 driver/firmware	Yes	Missing
Audio / input drivers	Yes	Missing

Metric	Pre-crash	Current
Win7/10/WinPE CAB packs	Yes	Missing

Full pre-crash file list: FactoryDocs/MANIFEST-pre-crash.txt (91 items flagged MISS).

Layout

Folder	Contents
System-T5810/	BIOS, chipset, ME, TPM, manuals
GPU/	AMD FirePro, NVIDIA (empty — re-download)
Storage/	PERC H710, Intel RST, SSD/HDD firmware
Network/	Intel Ethernet
Dell-Management/	Command Update, Configure
Expansion-Cards/	Serial, Thunderbolt docs
_Misc/	Non-Dell packages
_incoming/	Drop new Dell downloads here

How it is installed

Ingest new downloads:

```
cp ~/Downloads/* ~/Documents/IndianaDell/FactoryDocs/_incoming/
python3 ~/Documents/IndianaDell/FactoryDocs/_sort_factory_docs.py
```

Priority re-downloads from Dell T5810 drivers:

1. GPU/AMD-FirePro/Windows/ — Video_Driver_C5FPW (W5000/W5100)
2. Storage/RAID-Controller-PERC/Windows/ — PERC H710
3. System-T5810/Windows-10/ — T5810-win10 CAB
4. Audio/Windows/ — Audio_Driver_5P33P
5. Dell-Management/Windows/ — Command Configure, Monitor

Windows installs use Dell CAB/EXE packages from these folders — not apt.

How to verify

```
find FactoryDocs -type f ! -path '*/_incoming/*' | wc -l
cat FactoryDocs/README.md
grep MISS FactoryDocs/MANIFEST-pre-crash.txt | wc -l
```

How to customize

- `_sort_factory_docs.py` dedupes and sorts by hardware category

- Cross-reference hardware manual: B1GMB42-slot-port-inventory.md for what hardware needs drivers

What rebuild does / does not do

Does	Does not
Nothing	Copy FactoryDocs to system paths Install Windows drivers Download missing CABs

FactoryDocs recovery is a **manual** ongoing task (Chapter 3 item 7).

Chapter 14 — Gaps and Limits

Documented boundaries of what IndianaDell does **not** install or support on this host.

Not installed

Item	Notes
SDRangel, SigDigger	Not on Flathub; use gqrx + URH + inspectrum
Rust SDR crate workspace	Toolchain only — add crates per project with cargo add
ZFS / disk layout tools	Out of scope — handled at OS install time
Windows / dual-boot	FactoryDocs holds CABs; no auto-install
HackRF hardware test	No device attached at last verify (2026-07-05)
Ventoy full seed on every boot	Manual — run ~/bin/seed-ventoy-persistence.sh after changes
PNY rebuild-stick seed automation	Manual loop-mount of ubuntu-26.04.dat (see Ch. 15); stick units only fix groups/SSH/hostname

Lost in TPM/ZFS crash

Item	Recovery
Pre-crash full apt list	Not found — use <code>apt-full-manifest.txt</code> from current rebuild
Pre-crash FactoryDocs	82/101 packages missing — see <code>MANIFEST-pre-crash.txt</code>
Pre-crash apt-hamradio list	Regenerated by <code>rebuild save_manifests()</code>

GPU / ROCm matrix

Capability	W5000 / W5100 on Ubuntu 26.04
Desktop amdgpu	Yes — with etc/ configs
Vulkan (vkcube)	Yes
OpenCL (clinfo)	Yes (limited)
ROCm HIP / ML training	No — not in AMD ROCm support matrix for FirePro W5000/W5100

Use `bin/amd-preflight` before `bin/amd-install`; expect warnings for these GPUs.

Rebuild intentional omissions

These remain **manual** by design (see Chapter 3):

- `sudo bin/apply-amdgpu`
- `bin/apply-dark-mode`, `bin/apply-max-performance`, `bin/fix-nautilus-desktop-launch`, `bin/sync-desktop-icons`
- `bin/themes-extract`, `sudo bin/themes-install-boot`
- `bin/hackrf-flash-mayhem` (DFU with hardware)
- `bin/amd-install` (optional ROCm)

Future enhancement could fold GNOME/theme steps into rebuild; current manual documents the gap explicitly.

Encryption / TPM

Hardware manual recommends **not** re-enabling ZFS encryption until TPM + recovery strategy is documented. IndianaDell rebuild does not touch encryption.

ZFS boot import

The installed host **must** keep force import enabled:

```
# /etc/default/zfs
ZPOOL_IMPORT_OPTS="-f"
```

bin/rebuild-machine does **not** write this file. After reinstall or recovery, verify it yourself (Chapter 3 checklist). Without -f, boot can hang when pools need force-import (post-export hostid mismatch, unclean shutdown).

When something fails

1. Read scripts/rebuild/last-run.log
2. Run bin/rebuild-machine --verify-only for targeted MISS lines
3. Re-run individual phases (apt, HackRF build, Mayhem download) manually
4. Check chapter-specific verify sections

Reporting issues

Capture:

```
bin/dellmerge > debug.report
bin/rebuild-machine --verify-only 2>&1 | tee verify.log
uname -a && lsb_release -a
```

Chapter 15 — Ventoy Live Session & Persistence

This chapter covers **two** Ventoy volumes used on the lab:

Volume	Friendly name	Role	Typical size
Internal Seagate ST500DM002	Uncle Wiggly 🐰	Full rabbit hole: many ISOs, 24 GB Ubuntu persis- tence, DOS- BOOT / Win- dows / ISO- STASH	~466 GB disk

Volume	Friendly name	Role	Typical size
PNY USB 2.0 (~30 GB)	Rebuild stick	Lean Ubuntu 26.04 live + 3 GB persistence for recovery / thumper rebuild	~30 GB USB

Both use Ventoy: drop ISOs onto the data partition; they appear in the boot menu. Writable **casper-rw** persistence keeps login state across live boots.

EFI note (Tower5810): Uncle Wiggly's SATA port may be **disabled in Setup** for fast POST (docs/fast-boot.md). If /dev/disk/by-label/Wiggly is missing but the Seagate is cabled, re-enable that SATA port in **F2 Setup**, then reboot.

Overlay layout (both volumes)

Ubuntu live + Ventoy persistence uses an ext4 file labeled **casper-rw**:

```
persistence/ubuntu-26.04.dat # ext4, LABEL=casper-rw
upper/                        # modern overlay upper (preferred)
  home/<liveuser>/
  etc/
  usr/local/sbin/
  ...
work/                          # created by live system as needed
```

Older seeds may have used cow/upper/; current IndianaDell seeds use **upper/**. Seed scripts prefer upper/ and remove a stale cow/ tree when seeding.

Ventoy config (on the same data partition as the ISO):

```
{
  "persistence": [
    {
      "image": "/ubuntu-26.04-desktop-amd64.iso",
      "backend": "/persistence/ubuntu-26.04.dat",
      "autosel": 1
    }
  ]
}
```

```
]
}
```

Canonical repo copy for Uncle Wiggly: `scripts/ventoy/ventoy.json`. Paths are **absolute from the Ventoy data partition root**. Persistence backend and ISO must live on the **same** Ventoy data partition.

Subdirectories and non-ISO files

Ventoy **recursively scans** the data partition for bootable images (`.iso`, etc.). Subfolders are fine and already used on Wiggly:

Path example	Notes
<code>/perc/*.iso</code>	PERC FreeDOS/Linux kit; FreeDOS can use <code>auto_memdisk</code> in <code>ventoy.json</code>
<code>/Windows NT ISOs/*.iso</code>	Period OS media
<code>/QubesOS/Qubes-....iso + .asc + signing key</code>	Signatures sit next to the ISO; Ventoy ignores non-bootable files

Optional: empty `.ventoyignore` in a folder hides that tree from the boot menu.

Create a persistence image (example 3 GiB):

```
# From extracted Ventoy release, or scripts/ventoy/ if present
sudo ./CreatePersistentImg.sh -s 3072 -t ext4 -l casper-rw -o persistence/ubuntu-2
# Extend later:
sudo scripts/ventoy/ExtendPersistentImg.sh /path/to/ubuntu-26.04.dat <extra-MB>
# then resize2fs on the loop device if needed
```

Uncle Wiggly ☐☐ (internal Ventoy)

Names: friendly **Uncle Wiggly**; partition label **WiggLy** (historically `sdcl`, mount `/mnt/wiggly`). Same Seagate also holds **DOSBOOT**, Windows, **ISO-STASH** — see hardware inventory.

What gets persisted (full seed)

Item	Live path	In casper image
User home	/home/ubuntu (Wiggly full seed convention)	upper/home/ubuntu/
Installed packages	dpkg overlay	upper/var/lib/dpkg/
GDM autologin	/etc/gdm3/custom.conf	upper/etc/gdm3/
Grok auth + sessions	~/ .grok/	same (never in git)
GitHub CLI auth	~/ .config/gh/	same
SSH keys	~/ .ssh/	same
Chrome (tier C)	~/ .config/google-chrome/ curated	bookmarks, prefs, logins, Web Data, Extensions — no caches
Runtime secret source	/home/user/ when ZFS rpool is available	pulled via resolve-secrets.sh
IndianaDell workspace	~/Documents/IndianaDell	Full tree (git clone or rsync)
PATH overrides	~/ .config/indianadell	is a path.sh

Persistence image: /persistence/ubuntu-26.04.dat (**24 GB** ext4, label casper - rw) on label **Wiggly**.

One-time setup (Tower5810)

```
sudo mount -o uid=$(id -u),gid=$(id -g) /dev/disk/by-label/Wiggly /mnt/wiggly
bin/setup-wiggly-ventoy # verify ISO, ventoy.json, .dat filesystem
```

Seed session state

```
~/bin/seed-ventoy-persistence.sh
# or:
PERSIST_MOUNT=/mnt/persist-check ~/bin/seed-ventoy-persistence.sh
SEED_CHROME=c ~/bin/seed-ventoy-persistence.sh # default chrome tier
SEED_CHROME=off ~/bin/seed-ventoy-persistence.sh
```

Mode	When	Network?
Live casper overlay	Already booted from Ventoy persistence	No — local rsync only

Mode	When	Network?
External .dat seed	Seeding from Tower5810 / mounted volume	Only if IndianaDell must be git cloned

Network check: waits up to SEED_NETWORK_WAIT_SECS (default **120s**). Skip: SEED_SKIP_NETWORK_CHECK=1.

Chrome profile seed (SEED_CHROME)

Default **c**. Prefer /home/user/.config/google-chrome when rpool home exists; never copies Cache / Code Cache / GPU* / Service Worker.

Tier	What is copied
off / 0	Nothing
a	Bookmarks + Preferences
b	a + Local State + Secure Preferences
c	b + Login Data + Web Data + Extensions + Local Extension Settings
d	Reserved (same as c for now)

Login experience (Wiggly full seed)

1. **GDM autologin** — live user **ubuntu** (Wiggly seed convention; see rebuild stick for **user**)
2. **PATH** — IndianaDell bin/ and scripts/ via ~/.config/indianadell/path.sh
3. **Grok autostart** — often disabled (X-GNOME-Autostart-enabled=false)
4. **Installer** — no autostart; Desktop Install icon when needed

resolve-secrets.sh materializes secrets from /home/user when rpool exists, else Ventoy \$HOME.

PNY rebuild stick (lean USB)

Device: ~30 GB PNY USB (model often USB 2.0 FD). **Not** production storage — install/recovery only.

Why repartition / reinstall Ventoy

The old layout was ~6 GB Ventoy + 32 MB VTOYEFI + ~24 GB DOSBOOT. Ubuntu 26.04 desktop ISO is **~6.1 GB**, so the small Ventoy slice could not

hold ISO + persistence. Rebuild (2026-07) used Ventoy **1.1.16** force-install so almost the whole stick is one data partition:

Partition	Size	Label	Role
...1	~29.9 G	Ventoy	ISO + persistence + ventoy.json
...2	32 M	VT0YEFI	Ventoy EFI

```
# SAFETY: confirm USB transport + ~30 GB before -I
lsblk -o NAME,SIZE,MODEL,TRAN,LABEL
# From extracted ventoy-*-linux:
sudo bash ./Ventoy2Disk.sh -I -L Ventoy /dev/sdX # double-confirm y/y
```

Hiren's / old Ubuntu 22 / DOSBOOT content on that stick was wiped by reinstall. BartPE and current media live on **Uncle Wiggly** (or ISO-STASH).

Layout on the stick

```
/ubuntu-26.04-desktop-amd64.iso      # ~6.1 G (copied from Wiggly)
/persistence/ubuntu-26.04.dat        # 3 G casper-rw
/ventoy/ventoy.json                  # persistence map, autosel=1
```

Capacity rule of thumb: 6.1 G ISO + 3 G .dat = 9 G used; leave free space for future ISOs. A **24 GB** Wiggly-class overlay does **not** fit on this stick.

Lean seed policy (rebuild stick)

Include (as live user home — see identity below):

Item	Source (Tower5810)	Notes
SSH keys	/home/user/.ssh/	id_rsa, config, authorized_keys; mode 700/600
GitHub CLI	/home/user/.config/gh/	hosts.yml token
Grok	/home/user/.grok/	auth + sessions; prune downloads/ to save space optional
GnuPG	/home/user/.gnupg/	
gitconfig	/home/user/.gitconfig	
Cursor auth	~/.config/cursor/auth.json	if present
Shell dots	.bashrc, .profile, ...	
IndianaDell config	~/.config/indianadell, autostart, dconf	

Item	Source (Tower5810)	Notes
~/bin	host ~/bin helpers	
Keyrings	~/ .local/share/keyrings	
Project PDFs only	Documents/IndianaDell/B1GMB42-*.pdf	manuals / trifolds / ZFS recovery — not full Documents tree

Exclude: full Documents/, .cache, Chrome bulk, cargo/rustup/wine/googleearth, full .local (except keyrings).

Seed path in the image: upper/home/user/... (after username change below).

Mount / edit / unmount pattern:

```
DAT=/run/media/$USER/Ventoy/persistence/ubuntu-26.04.dat # or by-label Ventoy
MOUNT=/mnt/persist-usb
sudo mkdir -p "$MOUNT"
LOOP=$(sudo losetup --show -f "$DAT")
sudo mount "$LOOP" "$MOUNT"
# edit $MOUNT/upper/ ...
sync
sudo umount "$MOUNT"
sudo losetup -d "$LOOP"
```

Live user identity: user (not ubuntu)

Casper creates the live account from **/etc/casper.conf**. **FLAVOUR must be non-empty** or casper ignores USERNAME / HOST and substitutes the flavour string.

Seeded on the rebuild stick:

```
# /etc/casper.conf (in upper/)
export USERNAME="user"
export USERFULLNAME="User"
export HOST="thumper"
export BUILD_SYSTEM="Ubuntu"
export FLAVOUR="Ubuntu" # required for USERNAME/HOST to stick
```

Also:

File	Purpose
upper/etc/gdm3/custom.conf	AutomaticLoginEnable=true, AutomaticLogin=user

File	Purpose
upper/etc/sudoers.d/casper	ALL=(ALL) NOPASSWD: ALL
upper/home/user/	Seeded secrets + lean home (uid/gid 1000)

Casper's 15autologin / 25adduser / 44pk_allow_ubuntu all use \$USER-NAME, so PolicyKit and autologin follow casper.conf when FLAVOUR is set.

Boot unit: groups + rename safety net

Script: upper/usr/local/sbin/indianadell-live-user.sh

Unit: indianadell-live-user.service (WantedBy multi-user.target and graphical.target)

Log: /var/log/indianadell-live-user.log

Behavior:

1. If account **ubuntu** exists and **user** does not → groupmod/usermod rename + move home (old overlays / missed FLAVOUR).
2. Add **user** to every **special group that exists** on the live system (skip missing groups quietly).
3. Refresh sudoers + GDM autologin for user.
4. Set hostname **thumper** and 127.0.1.1 thumper.local thumper in /etc/hosts.

Group list (lab / rebuild relevant — install packages later may create more groups; reboot re-runs attach):

Category	Groups
Admin / desktop	adm, cdrom, sudo, dip, plugdev, lpadmin, sambashare, users
Serial / radio	dialout, tty, uucp
AV / GPU / input	audio, video, render, input
Print / scan	lp, scanner
Network	netdev, bluetooth
Containers / VMs	docker, lxd, kvm, libvirt, libvirt-qemu
Other	disk, floppy, ssl-cert, wireshark, fuse

Matches the intent of Tower5810's host user groups (id user) plus hardware-facing groups we may install on live.

Fast-boot deferments (Tower5810 parity + live snap kill)

Same idea as `bin/apply-fast-boot / docs/fast-boot.md`, adapted for **casper overlay** (cannot run `systemctl disable` on the ISO lower layer — use **mask symlinks**).

Mechanism	Path in overlay	Role
Boot masks	<code>etc/systemd/system/<unit> → /dev/null</code>	Unit cannot start early (present as soon as root mounts)
Permanent mask list	<code>etc/indianadell-live-mask.list</code>	Never auto-start (snap seed, NM-wait-online, ...)
Deferred list	<code>etc/indianadell-deferred.list</code>	Unmasked + <code>start --no-block</code> after <code>graphical.target</code>
Socket-lazy list	<code>etc/indianadell-socket-lazy.list</code>	<code>docker / cups / snapd / libvirt sockets only</code>
Early assert	<code>indianadell-live-fastboot.service</code> □ <code>usr/local/sbin/indianadell-live-fastboot.sh</code>	Re-mask/stop thrashers; <code>log /var/log/indianadell-live-fastboot.log</code>
Post-desktop start	<code>indianadell-deferred.service</code> □ <code>usr/local/sbin/indianadell-start-deferred</code>	After GDM; does not block desktop

Permanent masks (rebuild stick — the usual multi-minute hang):

- `snapd.seeded.service` (main culprit on Ubuntu live)

- snapd.autoimport.service, snapd.core-fixup.service, snapd.recovery-chooser-trigger.service, ...
- NetworkManager-wait-online.service

Deferred (masked at boot, started later in background): snapd.service, cloud-init, cups, bluetooth, avahi, ModemManager, docker/libvirt stack, apport/whoopsie, unattended-upgrades, etc. (full list on the stick).

snapd.service itself is only started after the desktop is up (and only if you unmask/start it); **seeded** stays masked so it never blocks login.

Verify on live:

```
systemctl is-enabled snapd.seeded.service    # expect: masked
systemctl is-active snapd.seeded.service    # inactive
systemd-analyze blame | head -20
sudo tail /var/log/indianadell-live-fastboot.log
```

Plymouth theme (indianadell)

Tower's custom theme is seeded into the overlay (~20 MB):

Item	Overlay path
Theme tree	usr/share/plymouth/themes/indianadell/ (from host /usr/share/plymouth/themes/indianadell)
Default alternative	etc/alternatives/default.plymouth □ .../indianadell.plymouth
Default symlink	usr/share/plymouth/themes/default.plymouth □ alternatives
Daemon config	etc/plymouth/plymouthd.conf □ Theme=indianadell

indianadell-live-fastboot.sh re-asserts the default theme once rootfs is up.

Caveat: The **very first** splash frames still come from the **ISO initrd** (stock spinner/BGRT). After casper mounts the persistence overlay, Plymouth uses Theme=indianadell for late boot / shutdown / any restart of plymouthd. Fully replacing early ISO splash would require rebuilding casper/initrd on the ISO (not done on this stick).

Install source on Tower: sudo bin/themes-install-boot (see Ch. 5). Stick seed copies the **installed** theme tree, not a separate rebuild of animation frames.

Hostname **thumper** / **thumper.local** and OpenSSH on LAN

Hostname: `casper.conf HOST=thumper` plus `/etc/hostname` and `/etc/hosts(thumper.local)`. Cloud-init can be steered with `upper/etc/cloud/cloud.cfg.d/99-thumper-hostname.cfg` if present.

SSH unit (optional seed): `thumper-lan-ssh.service` `upper/usr/local/sbin/thumper-lan-ssh.sh`

- Waits for network if needed; installs **openssh-server** (apt, or debs under `/var/cache/thumper-debs/` if cached).
- Drop-in `/etc/ssh/sshd_config.d/99-thumper-lan.conf`: listen `0.0.0.0 / : :`, pubkey + password auth, no root login, no empty passwords.
- Enables/starts `ssh.service` (disables socket-only activation if needed).
- Allows UFW OpenSSH if UFW is active; starts `avahi-daemon` when available for mDNS **thumper.local**.
- Log: `/var/log/thumper-lan-ssh.log`

Login:

`ssh user@thumper.local`

or LAN IP from the live session

Use the same SSH private key as Tower5810 (`id_rsa`); public key is in `~/.ssh/authorized_keys` on the stick. Live password is typically blank at the console (casper); SSH should use **keys** (empty passwords disabled for SSH).

MOTD hint (if seeded): `etc/update-motd.d/99-thumper-ssh`.

Secrets policy (**resolve-secrets.sh**)

Canonical secret relative paths (never commit to git):

`.ssh`
`.grok`
`.config/gh`

When ZFS `rpool /home/user` is present, it is the live secret **source**; Ventoy persistence is the portable **store** under the live home. Chrome seed is separate (`SEED_CHROME`).

ZFS recovery (**rpool + bpool**)

Manual: `docs/B1GMB42-zfs-recovery.md` + `B1GMB42-zfs-recovery.pdf` (repo root and DOSBOOT recovery kit).

Boot Ventoy Ubuntu live — **do not** use a broken installed system as root.

```
sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell          # or recovery kit path
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
sudo ./mount-rpool-recovery.sh chroot
# repair; then exit and umount both scripts
```

Before rebooting installed OS: /etc/default/zfs must set ZPOOL_IMPORT_OPTS="-f". Kernel one-shot: zfsforce=1.

Deploy kit: bin/deploy-dosboot-recovery (from Tower5810). PDF on the rebuild stick: ~/Documents/IndianaDell/B1GMB42-zfs-recovery.pdf.

GitHub repository

<https://github.com/webaugur/IndianaDell> (private)

```
bin/pull-repo --verify
bin/pull-repo --dragonsdr
bin/push-repo
```

HTTPS optional: INDIANADELL_REMOTE=... after gh auth login. Large FactoryDocs use **Git LFS**.

On the rebuild stick, after network is up: git clone git@github.com:webaugur/IndianaDell.git using seeded SSH keys.

How to verify

Uncle Wiggly (full)

```
findmnt / | grep -qE 'cow|overlay' && echo "persistence overlay active"
grep AutomaticLogin= /etc/gdm3/custom.conf
echo "$INDIANADELL_ROOT"
which dellmerge pull-repo push-repo grok
bin/pull-repo --verify
```

PNY rebuild stick

```
# Identity
whoami          # expect: user
id              # expect sudo,adm,plugdev,dialout,... as available
hostname; hostname -f  # thumper / thumper.local
grep AutomaticLogin=user /etc/gdm3/custom.conf
```

```

# Persistence
findmnt / | grep -qE 'cow|overlay' && echo "overlay ok"
test -f ~/.ssh/id_rsa && test -f ~/.config/gh/hosts.yml && test -f ~/.grok/auth.j
ls ~/Documents/IndianaDell/*.pdf

# SSH on LAN (after thumper-lan-ssh has run)
ss -tlnp | grep ':22'
systemctl is-active ssh
# from another host:
# ssh user@thumper.local

# Fast-boot / snapd not thrashing
systemctl is-enabled snapd.seeded.service # masked
systemd-analyze blame | head -15

# Plymouth (rootfs theme)
grep Theme= /etc/plymouth/plymouthd.conf # indianadell
readlink -f /etc/alternatives/default.plymouth

# Logs
sudo tail -50 /var/log/indianadell-live-user.log
sudo tail -50 /var/log/thumper-lan-ssh.log
sudo tail -50 /var/log/indianadell-live-fastboot.log

```

How to customize

Goal	Action
Re-seed Wiggly full session	~/bin/seed-ventoy-persistence.sh
Change Grok session	Edit GROK_SESSION_ID in grok-indianadell-launch.sh
Enlarge persistence	ExtendPersistentImg.sh + resize2fs
Verify Wiggly layout	bin/setup-wiggly-ventoy
Edit rebuild stick overlay	loop-mount ubuntu-26.04.dat, edit upper/, unmount
Live user name	upper/etc/casper.conf (USERNAME + FLAVOUR) + GDM + home dir + indianadell-live-user.sh
Extra groups	Edit SPECIAL_GROUPS in indianadell-live-user.sh
SSH / hostname	thumper-lan-ssh.sh + sshd drop-in + hosts/hostname

Goal	Action
Boot defer / mask lists	<code>etc/indianadell-deferred.list</code> , <code>etc/indianadell-live-mask.list</code>
Refresh Plymouth on stick	<code>rsync host</code> <code>/usr/share/plymouth/themes/indianadell/</code> into overlay; keep <code>plymouthd.conf</code>
Hide ISO folder from menu Qubes + signatures	<code>.ventoyignore</code> in that folder e.g. <code>Wiggly/QubesOS/*.iso + .asc +</code> key (subdirs OK)

Related tools

Tool	Role
<code>bin/setup-wiggly-ventoy</code>	Verify Wiggly ISO + <code>ventoy.json + .dat</code>
<code>bin/boot-uncle-wiggly-vm</code> <code>bin/boxes-import-wiggly-isos</code>	QEMU live + persistence smoke test GNOME Boxes domain per ISO on Wiggly
<code>bin/setup-perc-ventoy</code>	PERC FreeDOS/IT kit on Wiggly (perc/)
<code>~/bin/seed-ventoy-persistence.sh</code>	Full seed into casper image
<code>scripts/ventoy/resolve-secrets.sh</code>	Secret path policy + Chrome tiers
<code>scripts/ventoy/ventoy.json</code>	Canonical Wiggly Ventoy plugin config
Ventoy release <code>CreatePersistentImg.sh</code>	Build empty casper-rw .dat

What rebuild does / does not do

Does	Does not
Install Chrome, gh, git-lfs when run on a full live session	Auto-configure Ventoy <code>ventoy.json</code>
Document seed / stick layout in this chapter	Auto-run seed every boot (except stick units: groups/SSH)

Does	Does not
	Manage Seagate partition map (Wiggly/DOSBOOT/Windows) Fit a 24 GB Wiggly .dat on the 30 GB PNY stick

Appendix A — bin/ Launchers

All launchers live in `~/Documents/IndianaDell/bin/`. **PATH** is set automatically via `~/config/indianadell/path.sh` (IndianaDell tools override system binaries).

Launcher	Runs	Chapter
<code>rebuild-machine</code>	<code>scripts/rebuild-machine.sh</code>	2
<code>build-software-manual</code>	<code>scripts/docs/build-software-manual.sh</code>	
<code>build-all-docs</code>	<code>scripts/docs/build-all-docs.sh</code>	
<code>pull-repo</code>	<code>scripts/github/pull-all.sh</code> — IndianaDell + LFS (--dragonsdr optional)	15
<code>install-dragonsdr</code>	<code>~/Documents/DragonsDR/bin/install-suite</code> — full SDR suite	15
<code>push-repo</code>	<code>bin/push-repo</code> — GitHub webaur/IndianaDell (SSH default)	
<code>setup-wiggly-ventoy</code>	<code>scripts/ventoy/setup-wiggly-ventoy.sh</code> — Uncle Wiggly ISO + ventoy.json + .dat	15

Launcher	Runs	Chapter
setup-perc-ventoy	scripts/perc/setup-perc-ventoy.sh — H710 FreeDOS/IT kit on Uncle Wiggly	hardware / PERC doc
boot-uncle-wiggly-vm	scripts/ventoy/boot-uncle-wiggly-vm.sh — QEMU live+persistencetest	boot
boxes-import-wiggly-isos	scripts/ventoy/boxes-import-wiggly-isos.sh — Boxes VM per ISO on Wiggly	boxes
themes-preview-boot	Themes/scripts/plymouth-preview.py — safe Plymouth window	themes
apply-fast-login	scripts/gnome/apply-fast-login.sh — GRUB 0s + GDM autologin + face	apply
apply-fast-boot	scripts/gnome/apply-fast-boot.sh — strip crashkernel, defer daemons	apply
build-zfs-recovery-doc	scripts/docs/build-zfs-recovery-doc.sh	docs
build-trifold-slick	docs/sales/B1GMB42-trifold.html □ sales PDFs	sales
deploy-dosboot-recovery	scripts/recovery/deploy-to-dosboot.sh	recovery
efi-timing-suite	scripts/efi/efi-timing-suite.sh	efi, 12
dellmerge	scripts/dell/dellmerge.sh	dell

Launcher	Runs	Chapter
gpu-stress	scripts/gpu/gpu6, 12 stress.sh	
iotest	scripts/storage/12iotest.sh	
apply-amdgpu	etc/apply.sh	6
amd-install	amd- radeon/install- all.sh	6
amd-preflight	amd- radeon/00- preflight.sh	6
amd-verify	amd- radeon/04- verify.sh	6
amd-uninstall	amd- radeon/uninstall.sh	6
apply-dark-mode	scripts/gnome/apply- dark-mode.sh	7
apply-max-performance	scripts/gnome/apply- max- performance.sh	7
fix-nautilus-desktop- launch	scripts/gnome/fix7 nautilus- desktop- launch.sh	
sync-desktop-icons	scripts/gnome/sync7- desktop- icons.sh	
themes-extract	Themes/scripts/extract- all.sh	5
themes-install-boot	Themes/scripts/install- boot-theme.sh	5
themes-restore-boot	Themes/scripts/install- boot-theme.sh --restore- stock	
hackrf-env	sources Drag- onSDR/hackrf/scripts/env.sh	10
urh	DragonSDR/bin/urh	10
hackrf-setup-udev	DragonSDR/bin/hackrf- setup-udev	10
hackrf-download- mayhem	DragonSDR/bin/hackrf- download- mayhem	

Launcher	Runs	Chapter
hackrf-prepare-sdcard	DragonSDR/bin/hackrf-prepare-sdcard	10
hackrf-flash-mayhem	DragonSDR/bin/hackrf-flash-mayhem	10
hackrf-build-mayhem	DragonSDR/bin/hackrf-build-mayhem	10

Ventoy session (scripts/ventoy/ → ~/bin via install-ventoy-session.sh):

Script	Purpose
seed-ventoy-persistence.sh	Snapshot session into Ventoy casper image (Uncle Wiggly full seed)
seed-network-check.sh	Internet/DNS check before seed
grok-indianadell-launch.sh	Seed then Grok fullscreen autostart
install-ventoy-session.sh	Install helpers, autostart, PATH
mount-rpool-recovery.sh	ZFS rpool chroot recovery (workspace root)
mount-bpool-recovery.sh	ZFS bpool mount at /recovery/boot (scripts/recovery/)

PNY rebuild stick (lean 3GB persistence, live user user, groups, thumper/SSH): no dedicated bin/ launcher yet — procedure and overlay paths are documented in **Chapter 15** (upper/usr/local/sbin/indianadell-live-user.sh, thumper-lan-ssh.sh).

Note: hackrf-env must be **sourced**, not executed: source bin/hackrf-env

Sudo required: apply-amdgpu, themes-install-boot, themes-restore-boot, iotest, hackrf-setup-udev (udev install), amd-install, efi-timing-suite, mount-rpool-recovery.sh, mount-bpool-recovery.sh

Appendix B — Apt Packages by Chapter

Workstation packages: scripts/rebuild/package-lists.sh (APT_CORE only).

SDR / ham / HackRF packages: ~/Documents/DragonSDR/tools/package-lists.sh (APT_SDR, APT_HAM, APT_SDR_BUILD).

Install SDR suite: bin/install-dragonsdr □ DragonSDR bin/install-suite.

Full system snapshot: apt-full-manifest.txt (after rebuild).

SDR/ham filter snapshot: apt-hamradio-dev-manifest.txt.

Chapter 4 — Development (IndianaDell APT_CORE)

build-essential, cmake, pkg-config, git, curl, wget, unzip, python3-pip, python3-venv, python3-dev, python3-numpy, python3-scipy, python3-matplotlib, python3-yaml, python3-requests, python3-pyqt5, python3-psutil, libssl-dev, clang, llvm-dev, libclang-dev, libusb-1.0-0-dev, libfftw3-dev, libvolk-dev, portaudio19-dev, libsndfile1-dev, libboost-dev, libboost-program-options-dev, pandoc, texlive-latex-recommended, texlive-fonts-recommended, texlive-xetex, gh

Chapter 6 — GPU and Display

vulkan-tools, mesa-utils, mesa-utils-bin, clinfo

Chapter 8 — GNU Radio and SDR (DragonSDR APT_SDR + build libs)

gnuradio, gnuradio-dev, gnuradio-doc, gr-osmosdr, gr-limesdr, gr-fosphor, gr-air-modes, gr-hpsdr, gr-dab, gr-satellites, libsoapysdr-dev, python3-soapysdr, soapysdr-module-osmosdr, soapysdr-module-mirisdr, uhd-soapysdr, rtl-sdr, librtlsdr-dev, airspy, libairspy-dev, bladerf, libbladerf-dev, limesuite, limesuite-udev, uhd-host, libuhd-dev, gqrx-sdr, quisk, inspectrum, hacktv

Chapter 9 — Ham Radio (DragonSDR APT_HAM)

libhamlib-dev, libhamlib-utils, python3-hamlib, fldigi, wsjtx, wsjtx-data, chirp, direwolf, gpredict, grig, xastir, xastir-data

Chapter 10 — HackRF and Mayhem (DragonSDR APT_SDR)

hackrf, hackrf-firmware, libhackrf-dev, hackrf-doc, dfu-util, openocd, gcc-arm-none-eabi, binutils-arm-none-eabi, libnewlib-arm-none-eabi, ccache, lz4, bzip2

Chapter 11 — Flatpak

flatpak (application org.telegram.desktop installed via flatpak, not apt)

Chapters 5, 7, 12, 13 — No dedicated apt arrays

Themes, GNOME prefs, machine utilities, and FactoryDocs use workspace scripts or Ubuntu desktop packages already on the base install.